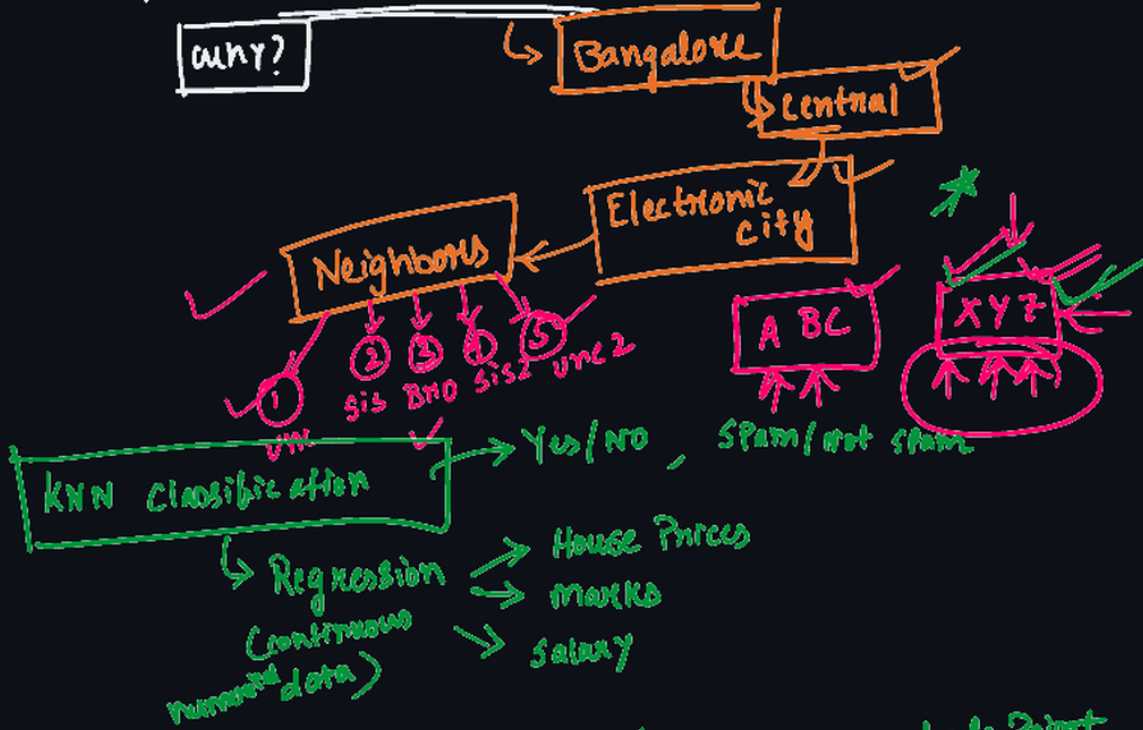


KNN → K - Nearest Neighbors



→ Predicting the answer for a new data point, by looking at the most similar existing data points.

KNN → **K** - Nearest Neighbors
 ↳ **No. of neighbors**

↳ with help of which I will be Predicting the answer for new data point.

Types of KNN:-

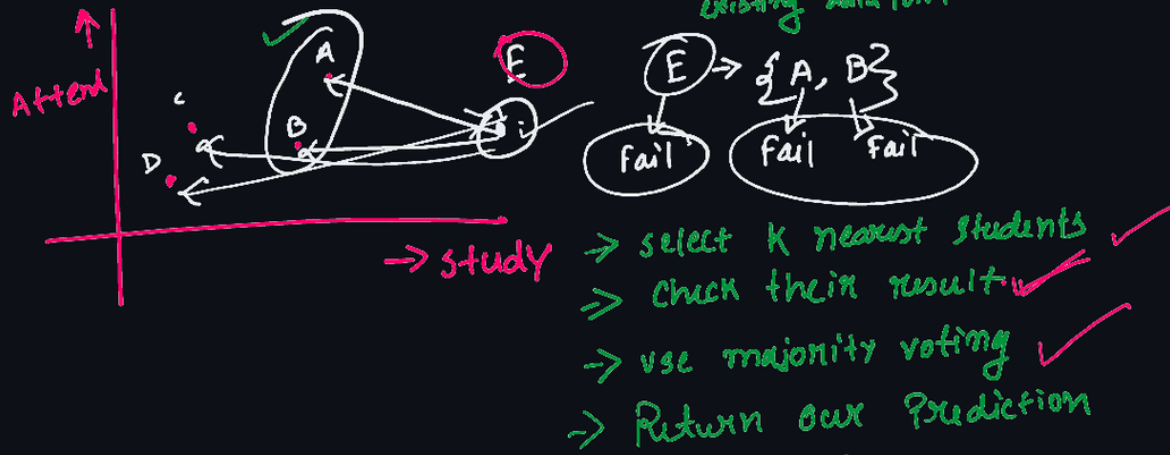
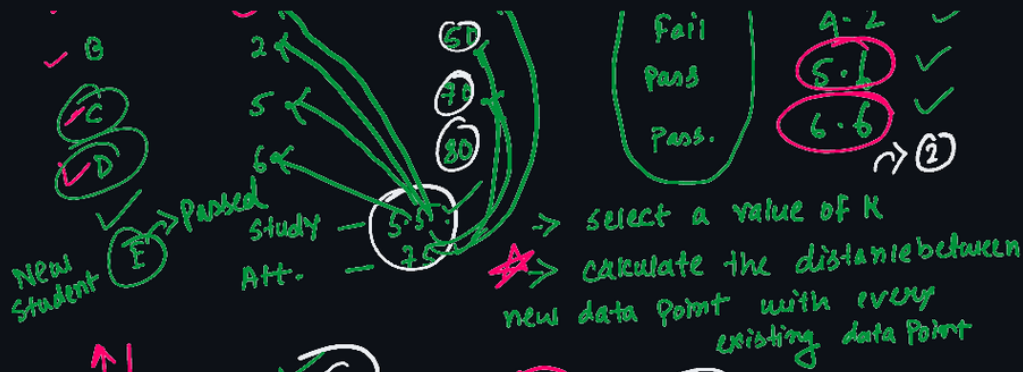
1) **Classification**
 → Spam/not spam
 → Passed/Failed
 (Categorical)

2) **Regression**
 → Predict House Prices
 → Salary.
 (Numerical)

why? → **KNN**

1) How KNN Classification Algorithm works?

	Study Hours	Attendance	Result	Score (CGPA)
✓ A	1	40	Fail	3.2 ✓
✓ B	2	50	Fail	4.2 ✓
✓ C	3	60	Pass	5.1 ✓



2) How KNN Regression Algorithm works?

$$E = \frac{5.6 + 6.6}{2}$$

To calculate distance from new data point to every existing Data Point:-

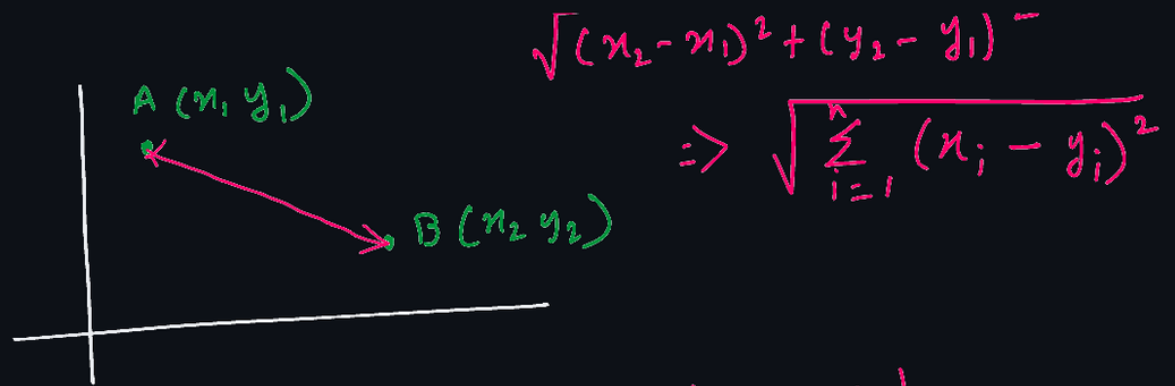
1) Euclidean Distance

2) Manhattan Distance.

$$d(A, B) =$$

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$A(x, y)$$



$$d(A, B) \Rightarrow |x_2 - x_1| + |y_2 - y_1|$$

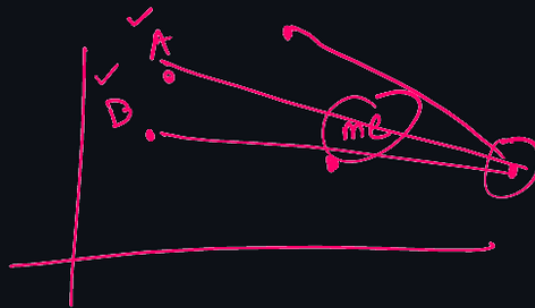
KNN $\Rightarrow$ Lazy Learner.

✓ 300 rows new

KNN $\Rightarrow$ K - Nearest Neighbors.
 $\hookrightarrow$ "How many nearest"

very small K :-

$\rightarrow$ Sensitive to distractions.



✓ South
 ✓ ABC

✓ North
 XYZ

K = 100

$\rightarrow$ Sensitive Noise

→ overfit

Best way to choose value:-

$$K = \sqrt{n}$$

$$K = \sqrt{n} = 10$$

$\therefore n =$ No. of samples.

Note :- way only for starting point.

weighted KNN

$$K = 3$$

distances

✓ 0.1

0.8

0.7

Predict

Fail

Pass

pass.

Pass ✓

Predict = Pass

weighted distances:

$$w = \frac{1}{d + \epsilon}$$

↓
Distance of nearest neighbor

Why feature scaling is important.

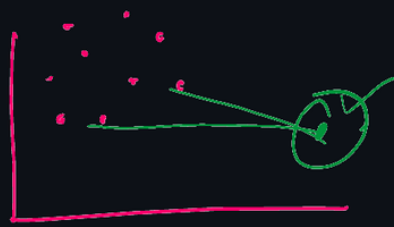
Why feature scaling is important.

A

B

Age
25 ✓
26 ✓

Salary
30000
80000



Manhattan

$$|x_2 - x_1| + |y_2 - y_1|$$

$$|26 - 25| + |80000 - 30000|$$

$$1 + 50000$$

model Parameter Tuning

n-neighbors $\rightarrow$ value of k

$\sqrt{n}$
 $\rightarrow$ cv technique.

$$\underline{\text{weights}} = \frac{\text{"uniform"}}{\text{"distances"}}$$

$$\text{metric} = \frac{\text{"manhattan"}}{\text{"Euclidean"}}$$

$\rightarrow$

$\rightarrow$

$\rightarrow$ Normally distributed

$\rightarrow$ Not having much outliers.

$\rightarrow$ Straight line.

algorithm = {
"brute" ✓
"kd-tree" ✓
"ball-tree" ✓
"auto" ✓

→ Not having many points
→ Straight line.

Brute Force



→ Dataset small

→ No. of columns is large.

KD-Tree

Divide the feature space into regions to search faster.

Ball-Tree

How to select the best value of k ?

→ $k = \sqrt{n}$

$\rightarrow K = \sqrt{n}$ $\hookrightarrow$ No. of samples.

Suppose

150 rows of data.

$\hookrightarrow$ 120 training 30 testing

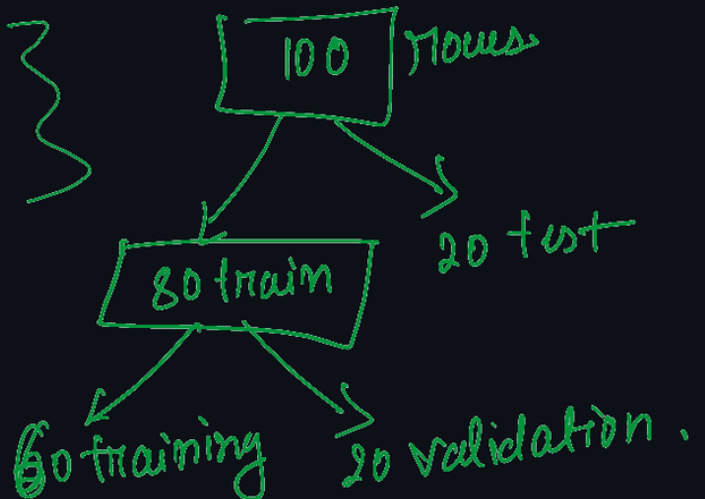
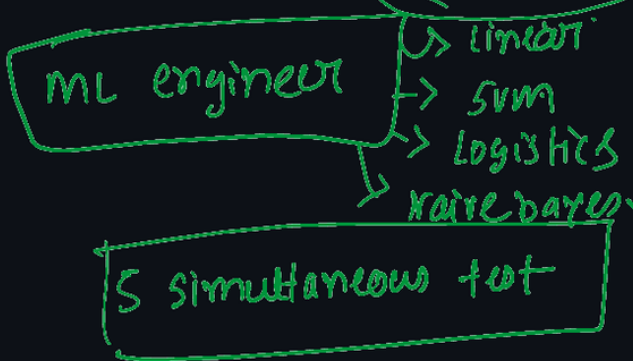
$K = 1, 2, 3, \dots$

Cross Validation

Taking the training dataset.

$\hookrightarrow$ Dividing training dataset into multiple parts.

Training vs testing vs validation $\hookrightarrow$ Train each parts $\hookrightarrow$ validate each part.



$k = 1, 2, 3 \dots$

⑤ bold cross validation →

5 Rounds of training
5 ^{random} groups of data.

$x_{\text{train}} = 120 \text{ points}$
↳ 5 groups

Fold 1 → 24 points

Fold 2 → 24 points

...

$k = 2$

Round 1

Fold 1 → Validation

Fold 2, 3, 4, 5 → Training → 95% (accuracy)

Round 2

Fold 2 → Validation

Fold 1, 3, 4, 5 → Training

→ accuracy 96%

95 + 96 + 92 + 98 + 94

$\sqrt{95\%}$

$200 \sqrt{200} \quad k = \sqrt{n}$

$k = \boxed{14}$ 95%
 $\frac{200}{\sqrt{200}}$

bon k in range $(1, \underline{16})$:

DF

<u>Age</u>	<u>Gender</u>	<u>BMI</u>	<u>Cancer</u>
23	m	102	Y
24	F	62	N
22	M	56	Y

$dh = \{ 'Age': [23, 24, 22], 'gender': ['m', 'F', 'F'] \}$